



LIV.INNO

Apache Spark: Introductory Workshop

Dr Alexander Hill, a.d.hill@liverpool.ac.uk

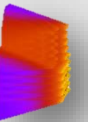
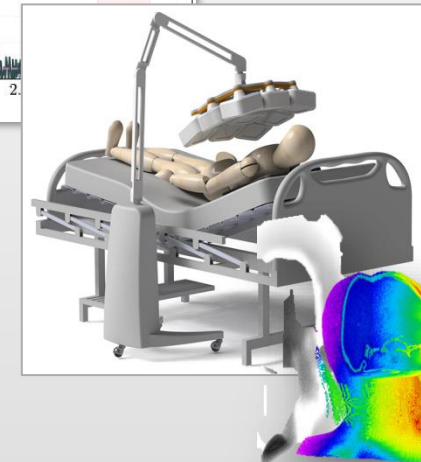
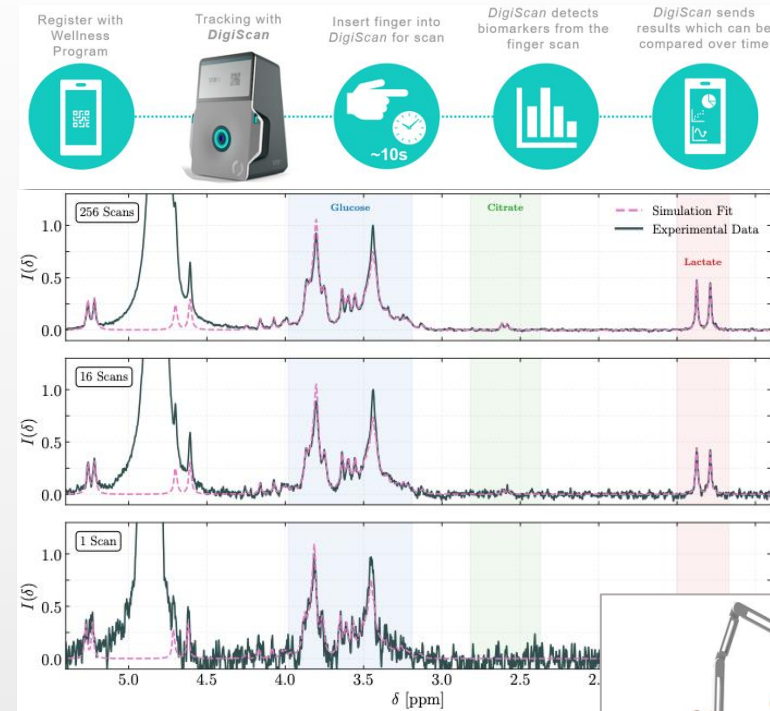
16th September 2026, Data Intensive Science School

Session overview

- Big picture context
- PySpark Introduction
- Hands-on activity with MLlib



My background



Big data challenges

How best to process, analyse, and store this data?



~10 TB every 24 hours: Vera C. Rubin Observatory

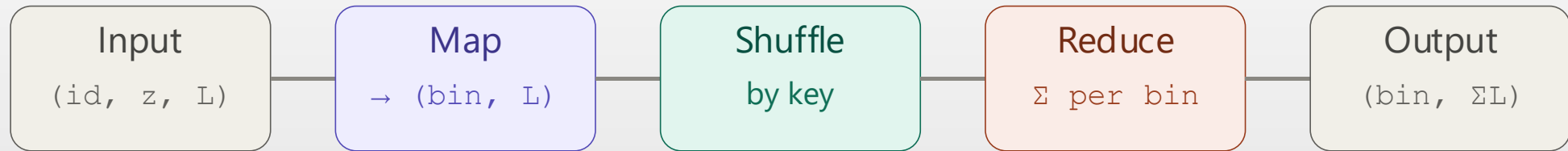


100s GB: Raw output of a single genome sequencing run



MapReduce and Hadoop

- 2004: Google publishes '*MapReduce: Simplified Data Processing on Large Clusters*'
- 2006: Apache Hadoop makes the system open source and widely usable



e.g. $g1(z=0.15, L=2.1) \rightarrow (bin_0, 2.1) \rightarrow \text{grouped with } g3 \rightarrow \Sigma=5.1$

- Fault tolerant and scalable
- However very expensive for iterative algorithms

Spark at UC Berkley

- 2009: UC Berkley PhD grad student Matei Zaharia and colleagues start the Spark project
- Addressed inefficiency for applications that reuse a dataset across multiple steps (e.g. iterative ML)
- Solution: Resilient Distributed Datasets (RDDs), a fault-tolerant, read-only abstraction of data split across a cluster
- Spark outperformed Hadoop by 10x in iterative ML jobs, and interactively queried a 39GB dataset with sub-second response time



Spark: Cluster Computing with Working Sets

Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, Ion Stoica
University of California, Berkeley

Research Project to Industry Standard



2010

RDD paper published

2013

Donated to the Apache
Software Foundation



databricks

2014

Apache top-level
project; Databricks
founded

2016+

DataFrames built on top of RDDs,
and linked with Spark SQL, mature
into the default API



Applications in Science



Particle physics

CERN experiments use Spark-based pipelines for some large-scale offline analysis and bookkeeping, alongside traditional ROOT/C++ tooling.



Genomics

Projects such as Hail (Broad Institute) build variant-calling and population-genetics pipelines on top of Spark.



Astronomy

Large sky-survey pipelines, at the scale of LSST/Rubin, use distributed processing for nightly data reduction.



Materials & chemistry

High-throughput virtual screening projects use distributed frameworks, including Spark, across massive compound libraries.



Large-facility computing and research software engineering

Industry Use Cases



~2 million Spark workloads per day in 2025 (real-time processing, ML, data analytics)



100TB of RAM, 2000 nodes, 20,000 cores used via YARN for e.g. providing targeted offers (2014)



30 million rows of data per hour processed to detect cheats in Call of Duty

Databricks

- Apache Spark is the core engine behind the Data + AI Platform
- Supports data engineering, data science and machine learning in one environment
- Allows organisations manage and analyse large amounts of their data
- Further allows organisations to build and train AI models and deploy analytics at scale



F PROFILE

Matei Zaharia

\$7B

Real Time Net Worth
as of 9/6/26
#582 in the world today

 databricks
DATA+AI SUMMIT

June 21-24, 2027 / San Francisco + Virtual

SEE YOU NEXT YEAR!

June 21-24, 2027 SAN FRANCISCO

WATCH ON DEMAND

WATCH THE 2025 SUMMIT RECAP



Spark Architecture

High-level libraries & APIs

Spark SQL

Spark
Streaming

MLlib

GraphX

Apache Spark Core

Cluster Resource Architecture

Standalone
Scheduler

YARN

Mesos

Spark's working languages



Python interface, however core execution is Scala and Java

How to get started with Spark



Local install

pip install pyspark - runs on your own machine using all its cores. Useful up to 10s of Gb. [First activity.](#)



Databricks Free Edition

Free, browser-based, serverless notebooks. [Second activity.](#)



Managed cloud clusters

AWS EMR, Google Dataproc, Azure Synapse.



Your institution's HPC cluster

Spark can run on Slurm or Kubernetes-managed clusters alongside traditional batch jobs.

Getting Started

- We will work with PySpark to get writing applications quickly
- Head to <https://github.com/Alex-Hill94/Spark> and clone/download the repository



Getting Started – Creating a Conda environment

create_env.sh

```
conda create -n SparkWorkshop \  
-c conda-forge \  
python=3.11 \  
ipython \  
numpy \  
pandas \  
pyspark \  
matplotlib \  
openjdk=17 \  
-y
```

test_env.sh

```
conda activate SparkWorkshop  
  
conda env list  
  
python -c "  
import numpy  
import pandas  
import pyspark  
import matplotlib  
  
print('All packages imported successfully!')  
print('PySpark version:', pyspark.__version__)  
"  
  
conda deactivate
```

BasicExample.py

```
from pyspark.sql import SparkSession
```

```
# Create a Spark session
```

```
spark = SparkSession.builder \  
    .appName("SimpleExample") \  
    .master("local[*]") \  
    .getOrCreate()
```

```
# Create some data
```

```
data = [  
    ("Alice", 25),  
    ("Bob", 17),  
    ("Charlie", 30),  
    ("Diana", 15)]
```

```
# Create a DataFrame
```

```
df = spark.createDataFrame(data,  
                            ["name", "age"])
```

```
# Filter adults
```

```
adults = df.filter(df.age >= 18)
```

```
# Display results
```

```
adults.show()
```

```
# Stop Spark
```

```
spark.stop()
```

```
+-----+----+  
|  name|age|  
+-----+----+  
|  Alice| 25|  
|Charlie| 30|  
+-----+----+
```

BasicExample.py

PySpark: Python API for Apache Spark.

SparkSession: main entry point for working with Spark DataFrames and other Spark features.



```
from pyspark.sql import SparkSession
```

BasicExample.py

SparkSession.builder: Starts a Spark application

appName: Names the application

master("local[*]"): Tells Spark to run locally on this machine

[*]: Use all available cores

getOrCreate: gets an existing SparkSession if one is already available, or creates a new one if necessary

```
# Create a Spark session
spark = SparkSession.builder \
    .appName("SimpleExample") \
    .master("local[*]") \
    .getOrCreate()
```

BasicExample.py

data: creates some dummy python data

spark.createDataFrame: creates a Spark DataFrame based on data, with headers name and age

```
# Create some data
data = [
    ("Alice", 25),
    ("Bob", 17),
    ("Charlie", 30),
    ("Diana", 15)]

# Create some data
df = spark.createDataFrame(data, ["name",
    "age"])
```

BasicExample.py

df.filter: Selects only rows satisfying a condition.

df.age >=18: Refers to the DataFrame column called "age", and selects only rows where entries are adults

```
# Filter adults  
adults = df.filter(df.age >= 18)
```

- This is not a python loop through the rows in Python, instead, it describes a filtering operation that Spark can (later) execute.
- This is a **transformation**. Spark transformations describe how to produce a new DataFrame from an existing one.

BasicExample.py

adults.show: Asks Spark to execute, performing enough work to produce and display the results

spark.stop(): Stops the Spark session and releases its resources.

Display results
adults.show()

Stop Spark
spark.stop()

BasicExample.py

- You might need to explicitly set the Spark driver binding address and host:

```
spark = SparkSession.builder \
    .appName("SimpleExample") \
    .master("local[*]") \
    .config("spark.driver.bindAddress", "127.0.0.1") \
    .config("spark.driver.host", "127.0.0.1") \
    .getOrCreate()
```

MoreDataExample.py

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MDEExample") \
    .master("local[*]") \
    .getOrCreate()

df = spark.range(0, 100_000_000)

values = df.filter(df.id % 3 == 0)

values.explain()

values.show()

counts = values.count()

spark.stop
```

MoreDataExample.py

df: Creates DataFrame object 'df', with 1e8 rows and one column, header 'id'

```
df = spark.range(0, 100_000_000)
values = df.filter(df.id % 3 == 0)
```

values: Creates the logic (a physical plan) for a new DataFrame object 'values', based on df, where rows are returned when id is divisible by 3

MoreDataExample.py

.explain(): A way of double checking the logic used to setup the values DataFrame

```
In [1]: print(counts)  
33333334
```

```
values.explain()  
  
values.show()  
  
counts = values.count()
```

.show(): Performs the 'show' action, i.e. printing out the DataFrame

.count(): Performs the 'count' action, counts the length of the values DataFrame

Data use



Spark is 'lazy'

- Spark uses lazy evaluation, meaning that data transformations (e.g. `.filter()`) are not executed immediately
- Instead, Spark records these as part of a logical execution plan, and only carries them out once an action (e.g. `.count()`) is triggered
- By delaying execution, Spark can analyse an entire sequence of transformations, and combine them into fewer passes over the data
- This further supports fault tolerance, Spark is able to recompute lost data by retracing the steps and re-executing failed stages

```
values = df.filter(df.id % 3 == 0)

values.explain()

values.show()

counts = values.count()

spark.stop
```

Data caching

- Spark remembers the plan associated with a DataFrame, but it does not automatically save outputs related to actions based on this
- We can use the `.cache()` action to force storage of intermediate data, which may be more efficient for multiple computations



Data caching

cache(): Marks the DataFrame adults for caching

```
adults = df.filter(df.age >= 18).cache()  
  
adults.count()  
  
# Second action: can reuse cached data  
adults.show()
```

Performs several actions:

- Execute filter on df to make adults DataFrame
- Caches the computed partitions
- Counts the length of 'adults'

Prints adults dataframe making use of the stored partitions

Data partitions

- `.master(local[*])` tells my machine to use all available CPU cores

```
import os  
print("Logical CPU cores:", os.cpu_count())
```

- When you create a DataFrame, Spark will plan to partition the data between available cores

```
print(df.rdd.getNumPartitions())
```

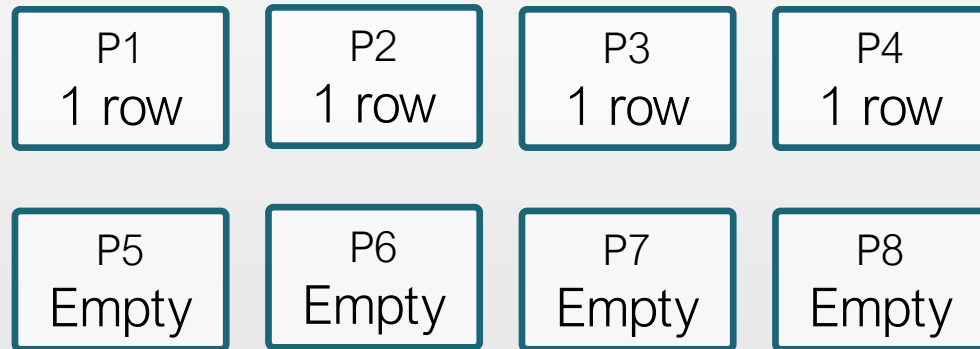
- These should be the same



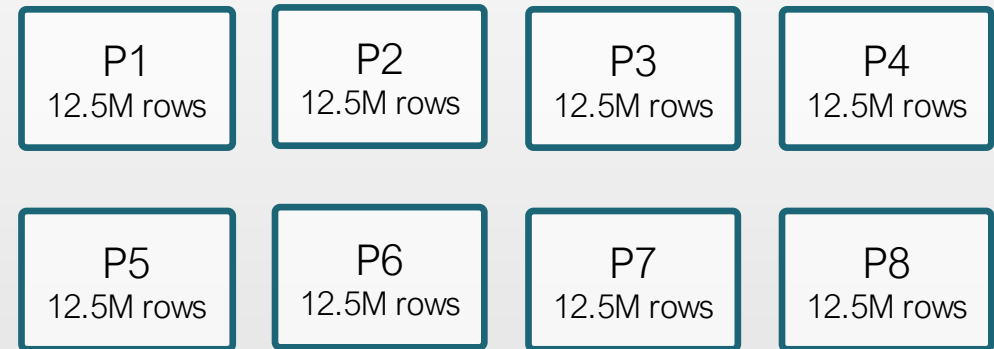
Data partitions

- Spark will likely as default set up the number of partitions as the number of logical cores available (e.g. 8)

Four rows, eight partitions



1e8 rows, eight partitions



Data Partitions

- You can set the number of partitions yourself using `.repartition(n)`, e.g.:

```
...: values = df.filter(df.id % 3 == 0)
...: values = values.repartition(16)
In [2]: values.rdd.getNumPartitions()
Out[2]: 16
```

- Having more partitions than cores available can make sense, particularly if one thread gets caught
- However, each partition has an overhead, so there will be diminishing returns



PySpark functionality



Functions.py

- Data handling functions in PySpark look very similar to SQL
- To get to grips with these, we'll work with Functions.py
- First we'll set up our data, then you can run exercise functions to test the outputs.

```
employees = spark.createDataFrame([    (1, "Alice",  
10, 60000),  
    (2, "Bob", 10, 50000),    (3, "Cara", 20,  
70000),    (4, "Dan", 30, 45000)],  
    ["employee_id", "name",  
    "department_id", "salary"])
```

Functions.py – Selecting and Transforming Columns

Grabs two columns from the DataFrame and prints them

```
employees.select("name", "salary")  
.show()
```

```
employees.select("name", "salary",  
                (F.col("salary") *  
                 1.1).alias("salary_with_raise"))  
.show()
```

Grabs two columns and prints them with a third, newly created column

N.b. this is not saved to memory

```
employees.select("name",  
                F.when(F.col("salary") >= 60000, "High")  
                  .otherwise("Standard").alias("salary_band"))  
.show()
```

Grabs the 'name' column and prints alongside a new column, salary_band

Functions.py – Selecting and Transforming Columns

```
employees_with_bonus = employees.withColumn(  
    "bonus", F.col("salary") * 0.10 )  
  
employees_with_bonus =  
    employees_with_bonus.withColumn(  
        "total_compensation", F.col("salary") + F.col("bonus")  
    )  
  
renamed = employees.withColumnRenamed(  
    "annual_salary", "salary"  
)  
  
renamed.drop("bonus")
```

Creates duplicate DataFrame with additional column **bonus**

Adds additional column to new DataFrame

Creates duplicate DataFrame with renamed column

Deletes column

Functions.py – Selecting and Transforming Columns

`select()` is used to select or create a specific set of columns, and the resulting DataFrame contains only those columns

`withColumn()` is used to add a new column or replace an existing column while retaining the other columns.



Functions.py – filtering

Filters based on one condition

```
employees.filter(F.col("salary") >= 60000  
)show()
```

```
employees.filter((F.col("salary") >= 60000)  
                & (F.col("department_id") == 10)  
)show()
```

Filters based on multiple conditions

& for AND
| for OR
~ for NOT

Activity

- Withing Functions.py, create a new DataFrame called 'dept10_weekly'
- This should contains columns ['employee_id', 'name', and 'weekly_salary'], only for employees of department 10
- Use as few computations as possible to get here



One solution

```
dept10_weekly =  
    (  
        employees  
        .filter(F.col("department_id") == 10)  
        .withColumn("weekly_salary",  
                    F.round(F.col("salary") / 52.0, 2))  
        .drop("salary", "department_id")  
    )
```

Functions.py – groupBy and agg

```
summary = employees.groupBy("department_id").agg(
    F.count("*").alias("employee_count"),
    F.avg("salary").alias("average_salary"),
    F.min("salary").alias("minimum_salary"),
    F.max("salary").alias("maximum_salary"),
    F.sum("salary").alias("total_salary")
).show()
```

No specific column needed, just asking for counts of occurrence of given dept_ids

Aggregate statistics make up new DataFrame

Shuffling

1. Rows initially distributed across existing partitions
2. `groupBy` causes a 'shuffle', by `department_id`
3. All rows for department 10 go together, All rows for department 20 go together, All rows for department 30 go together
4. Aggregate each group

Why shuffle? Efficient to have all relevant data in the same place (this is conceptually similar to MapReduce)

However, shuffling can be expensive so it's worth keeping an eye out for if you want to avoid it. Look for 'Exchange' in the Physical Plan.



Functions.py – join

- The `join` function enables the combination of datasets based on matched data (similar to vlookup in Excel)

```
In [56]: employees.show()
```

```
+-----+-----+-----+-----+
|employee_id| name|department_id|salary|
+-----+-----+-----+-----+
|      1|Alice|      10| 60000|
|      2| Bob|      10| 50000|
|      3| Cara|      20| 70000|
|      4| Dan|      30| 45000|
+-----+-----+-----+-----+
```

```
In [57]: departments.show()
```

```
+-----+-----+
|department_id|department_name|
+-----+-----+
|      10| Engineering|
|      20|      Sales|
|      30|        HR|
+-----+-----+
```

```
joined = employees.join(departments,
on="department_id", how="inner")
```

```
In [59]: joined.show()
```

```
+-----+-----+-----+-----+-----+
|department_id|employee_id| name|salary|department_name|
+-----+-----+-----+-----+-----+
|      10|      1|Alice| 60000| Engineering|
|      10|      2| Bob| 50000| Engineering|
|      20|      3| Cara| 70000|      Sales|
|      30|      4| Dan| 45000|        HR|
+-----+-----+-----+-----+-----+
```

Functions.py – join

- There are many ‘how’ conditions (inner, left, right, full etc), which relates to how join should behave in the instance of missing data or for where there are unmatched cases

```
joined = employees.join(departments,  
on="department_id", how="inner")
```

- For example ‘inner’ here only returns matching rows, ‘left’ would keep all employees even if a relevant department name is missing, ‘right’ is the opposite
- In practice here they would return the same DataFrame as the data is complete

Functions.py – join

- join can also combine datasets based on conditional logic, which for example is helpful if DataFrames don't have the same header

```
departments2 = departments.withColumnRenamed(  
    "department_id", "id")
```

```
joined = employees.join(departments2,  
    employees.department_id ==  
    departments2.id, how="inner"  
).show()
```

employee_id	name	department_id	salary	id	department_name
1	Alice	10	60000	10	Engineering
2	Bob	10	50000	10	Engineering
3	Cara	20	70000	20	Sales
4	Dan	30	45000	30	HR

Activity

- Create a new DataFrame called 'personnel_updated_salaries'
- Join **employees** with **departments** based on department_id
- Increase the salary of engineers by 20%, sales by 15%, and HR by 10%
- Drop all columns aside from name, department_name, and a new 'salary_updated' column



A solution

```
personnel_updated_salaries = (  
    employees.join(departments, on="department_id", how="inner") .withColumn("salary_updated",  
    F.when(F.col("department_name") == "Engineering", F.col("salary") * 1.20)  
    .when(F.col("department_name") == "Sales", F.col("salary") * 1.15)  
    .when(F.col("department_name") == "HR", F.col("salary") * 1.10)  
    .otherwise(F.col("salary")), )  
    .drop("department_id", "employee_id", "salary"))  
  
personnel_updated_salaries.show()
```

Diabetes prediction activity

- Now that we have some tools, we shall tackle a data cleaning problem
- In the MLlibActivity folder, open diabetic_risk_prediction.py
- This script takes synthetic medical data, and attempts to predict who has diabetes based on other factors
- This script uses MLlib's Logistic Regression functionality (binary classification)



Diabetes prediction activity

diabetes_data.csv

patient_id	age	height_cm	weight_kg	bp_systolic	bp_diastolic	glucose_mgdl	smoker	diabetic
1	65	167	72.8	143	94	-1	Yes	0
2	38	188	103.5	114	88	-1	Yes	1
3	76	177	76.6	106	65	156	Yes	1
4	41	179	51.3	105	95	166	No	0
5	48	195	124.2	105	74	144	No	1
6	78	173	64.1	138	70	88	No	0
7	42	190	unknown	153	63	153	Unknown	0
8	42	186	100.7	125	89	197	Yes	1

Diabetes prediction activity

How this activity works

1. Run this script. It will fail partway through (an exception about a missing column) because the data isn't ready yet (this is expected)
2. Complete the seven TODOs below using `select` / `withColumn` / `filter` / `groupBy+agg` / `join` etc.
3. Re-run the script. Once all TODOs are done, it should run end-to-end and print a trained logistic regression model with an AUC and accuracy score.
4. Try the "go further" suggestions at the bottom.

```
# TODO 2: Use .filter(...) to drop any row where height_cm or weight_kg is# now NULL  
(hint: F.col("height_cm").isNotNull()), and also drop any row where# glucose_mgdl == -  
1 (a dummy value indicating "not measured").
```

Diabetes prediction activity

Model evaluation:

- **AUC** represents the probability that the model ranks a randomly chosen positive instance higher than a randomly chosen negative instance
- **Accuracy** is simply $\text{Correct_Predictions} / \text{Total_Predictions}$



Takeaways



Transferable learning

- Think in terms of partitions and parallel work, not one sequential script.
- Separating “define the computation” from “execute” (laziness). This idea reappears elsewhere, e.g. ML computation graphs.
- SQL fluency: the querying pattern under DataFrames is the same one behind most data warehouses.
- Reasoning about when work needs data gathered together (a “shuffle”) versus when it can happen independently per chunk.



Why use Spark?

- Your data is genuinely multi-terabyte to petabyte scale, or will be soon
- You already have, or need, a multi-machine cluster
- You want one platform spanning batch SQL, streaming, and ML instead of stitching several tools together
- You're looking for an industry data science job, where Spark is a common expectation



When not to use Spark?

- You're prototyping and don't yet know if it'll ever need to scale. Start with e.g. pandas and migrate when you hit a wall.
- Data fits on one machine. Other solutions likely be faster to write and to run (Dask, Polars, or DuckDB).
- The job is mostly training/tuning ML models across many machines. Ray or a framework's native distributed training usually works better.
- You just need to query data someone else already manages. A SQL warehouse is simpler than standing up Spark yourself.



Next steps

- Lots of resources online (e.g. <https://sparkbyexamples.com/pyspark-tutorial/>; spark.apache.org/docs/latest/quick-start.html)
- Learning Spark, 2nd Edition — free e-book (Databricks)
- Databricks Free Edition sign-up



Let's Collaborate!

Partnership opportunities:

- PhD placements (3-6 months)
- Co-fund joint studentships
- Collaborative R&D projects
- Hackathons and workshops
- Access to new tech and talent pipelines

Stay connected: a.d.hill@liverpool.ac.uk
Quarterly newsletter: www.livinno.org

Developing people. Advancing science. Benefiting society.

